

튜링의 낙관

서동휘, KAIST 전산학부

이 글은 반복적인 작업을 기계에 맡길 수 있다는 튜링의 낙관을 바탕으로, 인공지능 코딩 요원이 가져올 변화를 논한다. 인공지능은 설계와 검증 등 창의적인 영역까지 자동화하지만, 그 결과를 사람이 검토하는 새로운 고역을 낳을 수 있다. 엄밀검증은 이러한 검토를 기계에 맡기고 인간이 프로그램이 만족해야 할 성질의 설계에 집중하도록 도울 수 있다. 미래의 프로그래밍에서 중요한 것은 자동화의 경계 너머에서 새로운 문제와 재미를 발견하는 일이다.

1946년 앨런 튜링은 영국 국립물리학연구소에서 범용 전자식 컴퓨터 ACE의 설계를 구상하고 있었다. 그는 이 기계를 실제로 작동시키기 위한 명령표(instruction tables)를 어떻게 작성할 것인지 설명하면서 다음과 같이 썼다 [1].

There need be no real danger of it ever becoming a drudge, for any processes that are quite mechanical may be turned over to the machine itself.

프로그래밍이 지루한 일이 될 위험은 없다. 완전히 기계적인 과정은 기계 자체에 맡길 수 있기 때문이다.

튜링에게 프로그래밍은 단순히 기계에 명령을 입력하는 일이 아니었다. 반복적인 부분을 자동화하면서, 인간은 그보다 더 흥미롭고 창의적인 문제를 다룰 수 있는 활동이었다.

돌이켜보면 프로그래밍의 역사는 상당 부분 이런 방식으로 진행되어 왔다. 초창기 프로그래머는 기계의 언어를 직접 다루어야 했지만, 어셈블러와 고급 언어 컴파일러가 등장하면서 수많은 세부 작업이 점차 기계 아래로 내려갔다. 중요한 점은 세부 작업을 직접 다룰 필요가 줄어들어도 프로그래밍 자체가 사라지지는 않았다는 것이다. 오히려 개발자가 고민해야 할 문제의 수준을 높였다. 명령어 하나하나에서 알고리즘으로, 알고리즘에서 시스템 설계로, 다시 사용자와 제품의 문제로 관심의 중심이 이동했다.

인공지능 코딩 요원(agent)은 이 오랜 흐름의 연장선에 있으면서도 이전과는 다른 성격을 가진다. 세부 작업을 대신 처리하던 기존 도구는 대체로 결정적(deterministic)이었다. 컴파일러는 미리 정해진 정교한 번역 절차를 따른다. 반면 인공지능 요원은 자연어로 주어진 모호한 요구를 해석하고, 코드를 수정하며, 실패하면 다른 전략을 시도한다. 결과는 예측할 수 없지만, 다룰 수 있는 범위는 전통적인 도구보다 훨씬 넓다. 이제 인간이 맡기는 일은 단순 명령어 입력을 넘어 버그 잡기(debugging), 설계, 검증, 심지어 현실 세계의 일부 의사결정까지 포함한다. 우리가 지금까지 ‘재미있다’고 여겨온 부분까지 건드리기 시작한 것이다.

그렇다면 자동화가 계속될수록 프로그래밍에서 인간의 몫은 줄어들기만 할까? 더 나아가 우리가 지금 프로그래밍의 재미라고 여기는 것 역시 사라질까? 튜링의 낙관을 끝까지 밀어붙이면 오히려 새로운 종류의 프로그래밍이 모습을 드러낸다. 인공지능이 만들어낸 코드를 사람이 일일이 읽고 오류를 찾아내는 일은 곧 또 다른 ‘지루한 일’이 될 수 있다. 이제 중요한 것은 프로그램이 반드시 만족해야 할 조건을 명시하고, 기계가 그것을 증명하게 만드는 것이다. 엄밀검증(formal verification)은 프로그램이 특정한 성질을 만족한다는 것을 수학적으로 기술하고 확인함으로써, 확률적으로 코드를 생성하는 인공지능에 결정적인 신뢰를 부여한다 [2]. 인공지능 시대에 새롭게 생겨난 ‘검토의 고역’을 다시 기계에게 넘기고, 사람이 코드의 세부 동작보다 프로그램이 만족해야 할 성질에 집중하도록 돕는 방법인 것이다.

프로그래밍의 흥미로움은 특정한 작업에 붙어 있는 것이 아니다. 인간이 천공 카드(punch card)를 뚫는 역할에서 문제 정의자로 변모하는 동안, ‘재미’의 위치도 자동화의 경계와 함께 움직여 왔다. 그리고 인공지능은 그 경계를 어느때보다도 멀리 밀어내고 있다. 미래의 프로그래밍은 지금의 코드 작성과 전혀 닮지 않을 수 있다. 그러나 80년 전 튜링의 낙관이 옳다면, 그것이 곧 프로그래밍이 지루해진다는 뜻은 아니다. 오히려 더 중요한 질문은 이것이다. 우리는 더 높은 수준의 문제로 빠르게 올라갈 준비가 되어 있는가. 그리고 그곳에서는 무엇을 새로운 프로그래밍의 재미라고 부르게 될 것인가.

References

- [1] Alan M. Turing. 1946. *Proposed Electronic Calculator*. Technical Report. National Physical Laboratory, Teddington, UK.
- [2] Kaiyu Yang, Gabriel Poesia, Jingxuan He, Wenda Li, Kristin Lauter, Swarat Chaudhuri, and Dawn Song. 2026. Formal Reasoning Meets LLMs: Toward AI for Mathematics and Verification. *Commun. ACM* 69, 3 (2026), 66–73. doi:10.1145/3750038